

Written Reflection

DevSecOps Pipeline Implementation — Acme Financial Services (OWASP Juice Shop reference build)

Methodology

I approached this engagement risk-first, not tool-first. Before writing any pipeline configuration, I wrote the Risk Brief, forcing myself to reason about business impact before reaching for tooling — the sequence a real consulting engagement demands, and the order the brief itself asked for. Only after that did I move into the four SAST/secrets/SCA/container gates, followed by the preventive branch-protection control, and finally the rollout plan for scaling to 40 services.

A deliberate methodological choice throughout: I validated every claim against the real, unmodified Juice Shop codebase rather than writing generic rules or assuming the scenario brief's premises were accurate. Both custom Semgrep rules were retargeted after grepping the actual source and finding my first draft (a literal-string JWT-secret pattern) didn't match how the real hardcoded key was structured in `lib/insecurity.ts`. Similarly, I found that Juice Shop's own Dockerfile already implements several hardening best practices (distroless base, multi-stage build, non-root user) that the challenge brief's premise assumed were absent — so my Dockerfile hardening work is an honest, incremental pass on real gaps, not a fabricated before/after.

Trade-offs and Decisions

Parallel gate execution over a staged pipeline.

All four scanning workflows trigger independently and run in parallel rather than being chained with needs : dependencies. This gives a developer feedback from all four gates within minutes instead of waiting on a sequential chain, at the cost of some wasted compute scanning a container image for a PR that might already be rejected for an unrelated SAST failure. For a cost-sensitive or very slow-build environment, I would restructure this into staged jobs so expensive stages only run once cheaper, faster gates pass.

Semgrep over CodeQL.

Faster setup, and I already had hands-on familiarity with Semgrep from prior source-review work, letting me spend more engagement time on rule quality and validation rather than tool onboarding.

Plain-text Actions logs over SARIF/GitHub Code Scanning integration.

I initially wired SARIF output into the SAST workflow for GitHub's native Code Scanning UI, but `pull_request_target`-triggered workflows execute using the workflow file from the base branch rather than the PR branch — a GitHub security measure that made troubleshooting unnecessarily complex for a solo engagement. I simplified to plain-text output printed directly to the Actions log, which is easier to demo live and removes an entire class of branch-scoping confusion, at the cost of losing inline PR annotations and the Security tab dashboard view.

Own repository instead of a fork.

Forks run Actions with a restricted `GITHUB_TOKEN` scope by default, which would have broken the secrets-scanning and SBOM-upload steps on PRs from the fork. Cloning into my own repository avoided this entirely, at the cost of needing to identify and remove several of Juice Shop's own upstream community-contribution workflows (a spam-detection bot, an auto-rebase workflow) that were tuned for an open-source project accepting outside PRs, not a private security engagement, and were actively auto-closing my own pull requests.

Challenges Encountered

A pipe silently masked a gate's failure.

My SAST workflow initially piped Semgrep's output through `tee` to save a copy to a file. In a shell pipeline, the exit code returned is the last command's, not the first's — `tee` succeeding masked Semgrep's failure, so the gate showed green despite real blocking findings existing. I caught it by noticing the job succeeding despite known findings, traced it to the pipe, and fixed it using Semgrep's own `--text-output` flag instead, which writes the file without swallowing the exit code.

A well-known placeholder credential produced a false negative.

My first Gitleaks demo used AWS's own documentation example key (`AKIAIOSFODNN7EXAMPLE`), which Gitleaks correctly ignores by design as a known, widely-used placeholder — flagging it everywhere would be pure noise. I switched to a randomized key matching the real AWS format to get a genuine true positive, a useful illustration of how detection tools balance thoroughness against false-positive fatigue.

Branch protection blocked even the repository owner.

While cleaning up an unrelated upstream workflow file directly on `master`, my own branch protection rejected the push with a `GH013` error. Rather than treating this as a blocker, I used it as stronger evidence than a settings screenshot could provide: the control was live and enforcing against everyone, including the person who configured it.

Gitleaks scans incrementally; Semgrep scans the full repository.

I discovered these two tools made opposite, deliberate default choices: Gitleaks only scans the commit range introduced by a given push (sensible, since a static historical commit doesn't change), while Semgrep re-scans the entire tracked file set every run (necessary, since a new or edited rule needs to be evaluated against existing code, not just new commits). This means a secret committed before Gitleaks was ever added to a repository would never be caught by ongoing push-triggered scans — a real gap I address in the rollout plan and revisit below.

What I Would Improve With More Time

A one-time full-history secrets scan as part of onboarding each of the 40 services, run before switching that service's Gitleaks gate to block mode, to close the incremental-scan blind spot described above.

Digest-pinning both Dockerfile base images, deferred here since it required a local `docker pull/inspect` step I did not have reliable access to during this engagement. **DAST** (OWASP ZAP) against a running Juice Shop instance, to cover runtime behavior neither SAST nor SCA can see. **IaC scanning** (Checkov/tfsec) if Terraform existed, plus a separate, non-CI-gated mechanism — a scheduled cloud-posture scan, not a pipeline gate — for catching account-level misconfigurations like the `AdministratorAccess` IAM roles flagged in the Risk Brief, since that is a live account-state problem a code-scanning pipeline structurally cannot see. Finally, given the scenario states there is no existing security team, I would treat the pilot phase of the rollout itself as the business case for a dedicated AppSec hire, rather than assuming the 3 existing DevOps engineers can absorb this work indefinitely alongside their existing responsibilities.